

KOENIG
step forward



Spring Boot



Core Spring Boot and Microservice Foundations



Introduction to Spring Boot and Microservices

Introduction to Spring Boot and Microservices



- Why microservices? Benefits and challenges
- Overview of Spring Boot and its role in microservice development
- Key components of a Spring Boot application



Why microservices? Benefits and challenges

- Microservices Architecture is a **design approach** where an application is divided into **small, independent services**.
- Each service:
 - Has its own database
 - Is independently deployable
 - Focuses on a single business capability
- Communication:
 - REST APIs
 - Messaging (Kafka, RabbitMQ)



Why microservices? Benefits and challenges

- Why Microservices?
 - Scalability (scale individual services)
 - Flexibility (different tech stacks)
 - Faster development by multiple teams
 - Fault isolation
- Microservices are not just a trend—they solve real problems in **large-scale, evolving applications**. But they also introduce new complexities.



Why microservices? Benefits and challenges

- **Benefits of Microservices**

- Independent Deployment

- Each service can be developed, tested, and deployed separately
- No need to redeploy the entire application

- Scalability (Fine-Grained)

- Scale only the services that need it
- Saves infrastructure cost

- Loose Coupling

- Services are independent
- Changes in one service don't break others (if APIs are stable)



Why microservices? Benefits and challenges

- **Benefits of Microservices**
- Team Autonomy
 - Multiple teams can work in parallel
 - Each team owns a service
- Technology Flexibility
 - Different services can use different technologies
- Fault Isolation
 - Failure in one service doesn't crash the entire system
- Faster Time to Market
 - Smaller codebases → faster development & releases



Why microservices? Benefits and challenges

- **Challenges of Microservices**
- Distributed System Complexity
 - Network calls instead of method calls
 - Issues:
 - Latency
 - Timeouts
 - Partial failures
- Difficult Debugging & Monitoring
 - Hard to trace requests across services
 - Requires:
 - Centralized logging
 - Distributed tracing



Why microservices? Benefits and challenges

- **Challenges of Microservices**
- Data Management Challenges
 - Each service has its own database
 - No simple joins across services
- Inter-Service Communication Overhead
 - REST or messaging adds overhead
 - Versioning APIs becomes critical
- DevOps & Infrastructure Complexity
 - Requires strong DevOps setup:
 - Docker
 - Kubernetes
 - CI/CD pipelines



Overview of Spring Boot and its role in microservice development

- Spring is a framework for Java application development, providing support for building scalable, reliable and easily maintainable application.
- Spring offers a variety of features that facilitate the development of Java applications.
- Spring is very popular framework for building Java Applications.
- The Problem with Spring
 - Building a traditional Spring application is really hard.
 - JAR dependencies, configuration, servers...



Overview of Spring Boot and its role in microservice development

- Spring boot is a project within the Spring framework designed to simplify the setup and development of new Spring applications.
- Make is easier to get started with Spring development
- Minimize the amount of manual configurations.
- Perform auto-configuration based of JAR dependencies.
- Spring Boot vs Spring
 - Spring Boot uses Spring behind the scenes.
 - Spring boot simply makes it easier to use Spring.



Overview of Spring Boot and its role in microservice development

- **Spring Boot** is a framework built on top of the Spring ecosystem that enables developers to create production-ready, standalone applications quickly with minimal configuration.
- Why Spring Boot?
 - Eliminates boilerplate XML configuration
 - Rapid development and easy setup
 - Standalone execution
 - Ideal for building REST APIs



Overview of Spring Boot and its role in microservice development

- **Core Capabilities of Spring Boot**

- Auto-Configuration
 - Automatically configures beans based on classpath dependencies
- Starter Dependencies
 - Simplified dependency management: spring-boot-starter-web
- Embedded Server
 - Runs applications without external deployment
 - Embedded Tomcat/Jetty
- External Configuration
 - Uses application.properties
- Production-Ready Features
 - Actuator (health checks, metrics)
 - Logging, monitoring hooks



Overview of Spring Boot and its role in microservice development

- **Key Advantages of Using Spring Boot for Microservices**
 - Fast development
 - Minimal configuration
 - Easy integration with cloud tools
 - Strong ecosystem (Spring Cloud)
 - Production-ready features
- Spring Boot simplifies development, but Microservices still require:
 - Proper design
 - DevOps maturity
 - Monitoring strategy

Key components of a Spring Boot application



- Main Application Class
 - This is the entry point of a Spring Boot application.
 - Annotation:
 - `@SpringBootApplication` (combines 3 annotations):
 - `@Configuration`
 - `@EnableAutoConfiguration`
 - `@ComponentScan`
- Starter Dependencies
 - Predefined dependency bundles to reduce manual setup.
 - Common Starters:
 - `spring-boot-starter-web` → REST APIs
 - `spring-boot-starter-data-jpa` → Database
 - `spring-boot-starter-security` → Security

Key components of a Spring Boot application



- Embedded Server
 - No need for external servers
 - Runs as standalone app
 - Tomcat (default)
- Spring Beans & Dependency Injection
 - Objects managed by Spring container = Beans
 - Common Annotations:
 - @Component
 - @Service
 - @Repository
 - @Controller / @RestController

Key components of a Spring Boot application



- Controller Layer (Web Layer)
 - Handles HTTP requests.
- Service Layer (Business Logic)
 - Contains business rules
 - Acts as a bridge between controller and repository
- Repository Layer (Data Access)
 - Handles database operations.
- Configuration Files
 - Externalizes configuration

Key components of a Spring Boot application



- Spring Boot Actuator
 - Provides production-ready endpoints:
 - /actuator/health
 - /actuator/metrics
 - /actuator/info
 - Used for monitoring and observability.



Key components of a Spring Boot application

- Common Stereotype Annotations

Annotation	Layer/Use Case	Inherits From	Description
@Component	Generic Spring-managed class	Root stereotype	Base annotation for any bean
@Service	Business logic layer	@Component	Marks a class as a service provider
@Controller	Web MVC controller (UI layer)	@Component	Handles web requests (returns view)
@RestController	RESTful API layer	@Controller	Returns JSON/XML directly



MVC Architecture Essentials



MVC Architecture Essentials

- Understanding the Model-View-Controller pattern
- Applying MVC principles to microservices
- Structuring a Spring Boot application for clarity and maintainability



Understanding the Model-View-Controller pattern

- MVC (Model–View–Controller) is a **design pattern** that separates an application into three main components:
 - **Model** → Data & business logic
 - **View** → UI (what user sees)
 - **Controller** → Handles requests & coordinates between Model and View
- Goal: Separation of concerns (clean, maintainable code)



Understanding the Model-View-Controller pattern

- Model (Data + Business Logic)
 - Represents application data
 - Contains business rules
 - Interacts with database
- View (Presentation Layer)
 - Displays data to the user
 - No business logic
- Controller (Request Handler)
 - Receives user requests
 - Calls business logic (Model)
 - Returns response (View)



Applying MVC principles to microservices

- In a **monolith**, MVC is applied **once** across the whole app.
- In **microservices**, **each service is a mini-application** → each one follows MVC independently.
- Key Principle:
 - Every microservice = its own **Controller + Service + Repository + Model**



Applying MVC principles to microservices

- **Design Principles**
- Single Responsibility per Service
 - Each microservice should represent one business capability
- API-First Controllers
 - Controllers expose clean REST APIs
- Thin Controllers, Fat Services
 - Controller → only handles request/response
 - Service → contains all business logic
- Database per Service
 - No shared DB across services
 - Each service owns its data



Applying MVC principles to microservices

- **Design Principles**
- Inter-Service Communication
 - Avoid direct repository access
 - Use:
 - REST APIs
 - Messaging (Kafka, RabbitMQ)

Structuring a Spring Boot application for clarity and maintainability

- A well-structured Spring Boot project:
 - Improves readability
 - Enables team collaboration
 - Simplifies testing & debugging
 - Scales easily for microservices
- Recommended Package Structure (Layered)
 - controller → REST APIs
 - service → Business logic
 - repository → Data access
 - model → Entities / domain objects
 - dto → Data Transfer Objects
 - exception → Custom exceptions
 - config → Configuration classes
 - util → Utility classes



Thymeleaf Basics

- Hyperlink

```
<a th:href="@{/dashboard}">Go to Dashboard</a>
```

- Display Message

```
<h1 th:text="${message}">Default Message</h1>
```

- Conditional Rendering

```
<p th:if="${isAdmin}">Welcome, Admin!</p>
```

```
<p th:unless="${isAdmin}">Welcome, User!</p>
```

- Iterate over a List

```
<ul>
```

```
<li th:each="emp : ${employees}" th:text="${emp.name}">Name Here</li>
```

```
</ul>
```



Microservice for Code Verification



Microservice for Code Verification

- Requirements gathering: Defining expected criteria for code verification
- Designing the service: Input validation, business logic, and result evaluation
- Implementing RESTful APIs for microservice communication



Requirements gathering: Defining expected criteria for code verification

- **Functional Requirements**
- Code Generation
 - Generate Numeric OTP (e.g., 6 digits)
 - Associate with userId or identifier (email/phone)
- Code Delivery
 - Send via SMS / Email
 - Integrate with providers like SMS gateways / Email services
- Code Validation
 - Verify: Correct code, Not expired, & Not already used
 - Return: SUCCESS / FAILURE



Requirements gathering: Defining expected criteria for code verification

- **Functional Requirements**
- Expiration & Retry Logic
 - Code expires after (e.g.) 5 minutes
 - Limit attempts (e.g., max 3 tries)
 - Resend support with cooldown
- Audit & Logging
 - Store:
 - Attempts
 - Status (verified/expired/failed)
 - Useful for fraud detection



Designing the service: Input validation, business logic, and result evaluation

- Input Validation
 - Validate: Email/phone format & Channel type (SMS/EMAIL)
- Business Logic
 - OTP Generation - Use secure random generator
- Result evaluation
 - Check: Code match, Expiration, & Attempt count



Implementing RESTful APIs for microservice communication

- OTP Generator

```
@Component
public class OtpGenerator {

    private final SecureRandom random = new SecureRandom();

    public String generate() {
        return String.valueOf(100000 + random.nextInt(900000));
    }
}
```



Verification, Validation, and MVC Architecture



Verification and Validation Techniques



Verification and Validation Techniques

- Writing unit tests for core functionalities
- Leveraging Spring Boot tools for input validation and error handling
- Monitoring and logging verification results



Writing unit tests for core functionalities

- Testing Stack
 - For a Spring Boot project, use:
 - JUnit 5 → Testing framework
 - Mockito → Mock dependencies
 - Spring Boot Test → Integration support



Writing unit tests for core functionalities

- What to Test (Core Functionalities)
 - OTP Generation
 - OTP is generated
 - OTP is stored in DB
 - Notification is triggered
- OTP Verification
 - Correct OTP → SUCCESS
 - Wrong OTP → FAIL
 - Expired OTP → FAIL
 - Max attempts exceeded → BLOCK



Leveraging Spring Boot tools for input validation and error handling

- Spring Boot provides:
 - Bean Validation → Input validation
 - `@ControllerAdvice` → Global error handling
- Together they ensure:
 - Clean APIs
 - Secure responses
 - Maintainable code



Monitoring and logging verification results

- Use Spring Boot Actuator
 - Spring Boot Actuator provides endpoints for monitoring.
- Use a Logging Framework
 - Spring Boot supports popular logging frameworks like: Logback (default)
- Best Practices
 - Use appropriate levels:
 - INFO → successful validation
 - WARN → minor validation issues
 - ERROR → critical failures



Monitoring and logging verification results

- Effective monitoring and logging of verification results involves:
 - Clear and structured logging
 - Centralized log management
 - Metrics collection via Actuator & Micrometer
 - Visualization dashboards
 - Real-time alerting



Hands-on Development



Hands-on Development

- Building a microservice to validate generated codes
- Applying constraints and business logic checks
- Deploying and testing the microservice in a simulated environment



Best Practices and Real-World Challenges



Best Practices and Real-World Challenges

- Ensuring scalability and performance in Spring Boot microservices
- Addressing common pitfalls in MVC-based microservice design
- Evolving microservices to meet new verification requirements

Ensuring scalability and performance in Spring Boot microservices



- Design for Statelessness
 - Keep services stateless → no session data in memory
 - Store state in external systems (DB, cache like Redis)
 - Why it matters: enables horizontal scaling (multiple instances behind a load balancer)
- Use Caching Strategically
 - Cache frequently validated codes (e.g., promo codes)
 - Reduces DB/API calls
 - Improves latency significantly

Ensuring scalability and performance in Spring Boot microservices



- Load Balancing & Scaling
 - Deploy multiple instances via Docker
 - Orchestrate with Kubernetes
 - Auto-scaling based on CPU/traffic
 - Fault tolerance
- Monitor Performance Metrics
 - Use:
 - Spring Boot Actuator
 - Prometheus
 - Grafana



Addressing common pitfalls in MVC-based microservice design

- Fat Controllers
 - Problem: Business logic inside controllers
 - Fix: Move logic to service layer
- Tight Coupling Between Layers
 - Controller → Service → Repository tightly bound
 - Fix:
 - Use interfaces
 - Apply dependency inversion



Addressing common pitfalls in MVC-based microservice design

- Overusing Synchronous Calls
 - Blocking REST calls between services
 - Fix: Use async messaging (Kafka/RabbitMQ)
- Poor Validation Strategy
 - Only validating at controller level
 - Fix:
 - Layered validation:
 - Input validation (DTO)
 - Business validation (service)



Addressing common pitfalls in MVC-based microservice design

- Ignoring Error Handling Standards
 - Inconsistent error responses
 - Fix: Centralized exception handling (@RestControllerAdvice)
- Shared Database Across Services
 - Breaks microservice independence
 - Fix: Database per service pattern



Evolving microservices to meet new verification requirements

- Real systems always change. Here's how to stay flexible.
- Instead of hardcoding:
 - `if (code.startsWith("PROMO"))`
 - Externalize rules:
 - DB
 - Config server
 - Feature flags
- Version Your APIs
 - When validation logic changes:
 - `/v1/validate`
 - `/v2/validate`
 - Avoid breaking clients



Introduction to REST API Principles



Understanding REST Architecture



Understanding REST Architecture

- Principles of REST: Statelessness, client-server separation, uniform interfaces.
- HTTP methods: GET, POST, PUT, DELETE, PATCH, and their appropriate use cases.
- API design best practices: Resource naming, versioning, and HATEOAS.



What is REST Architecture?

- **REST (Representational State Transfer)** is an architectural style for designing networked applications.
- It is based on a set of principles that enable communication between **clients** (such as web browsers or mobile apps) and **servers** over the **internet**.
- RESTful systems use **standard HTTP methods** to perform operations on resources, making them simple, scalable, and stateless.
- The three key principles of REST are Statelessness, Client-Server Separation, and Uniform Interfaces.



Principles of REST: Statelessness, client-server separation, uniform interfaces.

- **Statelessness**

- Each request from a client to a server must contain all the necessary information to process it.
- The server does **not store any client-specific session data** between requests.

- **Key Characteristics of Statelessness:**

- No session storage on the server.
- Each request is independent and self-contained.
- Clients must include authentication details (like API keys, JWT tokens) with every request.
- Improves scalability since the server does not maintain session state.



Principles of REST: Statelessness, client-server separation, uniform interfaces.

- **Client-Server Separation**

- The client (frontend) and server (backend) operate independently.
- The client sends requests, the server processes them and responds with data.

- **Key Characteristics of Client-Server Separation:**

- **Independence:** Clients and servers can evolve separately.
- **Scalability:** Multiple clients (web, mobile) can consume the same REST API.
- **Security:** Servers enforce authentication and authorization, keeping client-side logic separate.

- **Example Architecture:**

- Client (React, Angular, iOS, Android) → Requests Data
- Server (Node.js, Spring Boot) → Processes Requests and Returns Data



Principles of REST: Statelessness, client-server separation, uniform interfaces.

- **Uniform Interface**

- A REST API should have a consistent and standardized way of accessing resources, making it predictable and easy to use.

- **Key Constraints of Uniform Interface:**

- Resource Identification: Every resource has a unique URI (e.g., /users/1).
 - Standard HTTP Methods: RESTful APIs use:
 - GET → Retrieve data
 - POST → Create a new resource
 - PUT → Update an existing resource
 - DELETE → Remove a resource
 - Self-descriptive Messages: Responses should contain sufficient information for the client to process them.



HTTP methods: GET, POST, PUT, DELETE, PATCH, and their appropriate use cases.

- HTTP methods define the actions that can be performed on resources in a RESTful API.
- The most commonly used methods are GET, POST, PUT, DELETE, and PATCH.
- **GET – Retrieve Data**
 - Fetch data from the server without modifying it.
 - Use Cases:
 - Retrieve a list of resources (GET /products)
 - Retrieve a single resource (GET /products/1)



HTTP methods: GET, POST, PUT, DELETE, PATCH, and their appropriate use cases.

- **POST – Create a New Resource**

- Submit data to the server to create a new resource.
- Use Cases:
 - Register a new user (POST /users)
 - Add a new product (POST /products)

- **PUT – Update an Existing Resource (Full Update)**

- Replace an existing resource entirely with new data.
- Use Cases:
 - Update user profile (PUT /users/10)
 - Change product details (PUT /products/1)



HTTP methods: GET, POST, PUT, DELETE, PATCH, and their appropriate use cases.

- **PATCH – Update a Resource (Partial Update)**

- Modify specific fields of an existing resource instead of replacing the entire resource.
- Use Cases:
 - Update only the price of a product (PATCH /products/1)
 - Modify user details without affecting other fields (PATCH /users/10)

- **DELETE – Remove a Resource**

- Delete an existing resource from the server.
- Use Cases:
 - Remove a user account (DELETE /users/10)
 - Delete a product from the inventory (DELETE /products/1)



API design best practices: Resource naming, versioning, and HATEOAS.

- A well-designed REST API is easy to use, scalable, and maintainable.
- The three key best practices for API design are **resource naming, versioning, and HATEOAS** (Hypermedia as the Engine of Application State).



Resource Naming Best Practices

- Proper resource naming ensures clarity and consistency in API endpoints.
- Best Practices for Naming Resources:
 - Use **nouns**, not verbs (Resources represent entities, not actions).
 - Use **plural names** for collections and singular for specific resources.
 - Use **hyphens (-)** to separate words, not underscores (GET /user-profile, not GET /user_profile).
 - Use **lowercase** letters for URLs (GET /users, not GET /Users).



Resource Naming Best Practices

- Examples of Well-Designed Resource Names

Resource	Correct	Incorrect
Users Collection	GET /users	GET /getUsers
Single User	GET /users/10	GET /users?id=10
User's Orders	GET /users/10/orders	GET /users/orders?id=10
Search (with query parameters)	GET /products?category=electronics	GET /searchProducts/electronics



API Versioning Best Practices

- Versioning allows you to introduce new features or modify existing ones without breaking client applications.
- Best Practices for Versioning
 - Use **explicit versioning** in the URL (/v1/, /v2/), not implicit.
 - Use **major versions** only (e.g., v1, v2), not minor versions (v1.1).
 - Provide **backward compatibility** when updating APIs.
 - Deprecate old versions with a clear timeline.



API Versioning Best Practices

- Common API Versioning Methods

Versioning Method	Example	Pros	Cons
URL Versioning	GET /v1/products	Easy to implement, clear	Requires updating URLs when version changes
Header Versioning	GET /products + Accept: application/vnd.company.v1+json	Clean URLs, flexible	Clients must send correct headers
Query Parameter Versioning	GET /products?version=1	Easy for testing	Messy, less commonly used

- **Recommended: URL versioning (/v1/)** for clarity and ease of use.



HATEOAS (Hypermedia as the Engine of Application State)

- HATEOAS enhances REST APIs by including links in responses, guiding clients on available actions.
- Best Practices for HATEOAS
 - Include links to related resources in API responses.
 - Use self-descriptive links (self, update, delete).
 - Reduce the need for hardcoded URLs in client applications.



HATEOAS (Hypermedia as the Engine of Application State)

- Example of a HATEOAS-Enabled Response

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 1200,  
  "links": {  
    "self": "/products/1",  
    "update": "/products/1/update",  
    "delete": "/products/1/delete"  
  }  
}
```

- Benefits: Clients dynamically discover available actions instead of relying on documentation.



Building REST APIs with Spring Boot

- Setting up a Spring Boot project.
- Creating RESTful endpoints and mapping HTTP methods to Java methods.
- Handling request parameters, path variables, and request bodies.



Setting up a Spring Boot project.

1. Go to Spring Initializer: <https://start.spring.io/>
2. Select Project Settings:
 - Project: Maven
 - Language: Java
 - Spring Boot Version: Choose the latest stable version
 - Packaging: Jar
 - Java Version: 17 / 21
3. Define Project Metadata:
 - Group: com.example
 - Artifact: my-spring-boot-app
 - Package Name: com.example.my_spring_boot_app



Setting up a Spring Boot project.

4. Select Dependencies:

- Spring Web (for REST APIs)
- Spring Boot DevTools (for auto-reloading)

5. Generate & Download the Project:

- Click "Generate", then extract the ZIP file.

6. Open the Project in IntelliJ IDEA

- Open IntelliJ IDEA
- Click "Open" and select the extracted my-spring-boot-app folder.
- Wait for Maven dependencies to download.

Creating RESTful endpoints and mapping HTTP methods to Java methods.

- Create a Spring Boot REST Controller
 - Spring Boot uses the `@RestController` annotation to define RESTful APIs.

HTTP Method	Java Annotation	Example URL
GET	<code>@GetMapping</code>	<code>/users</code>
GET (by ID)	<code>@GetMapping("/{id}")</code>	<code>/users/1</code>
POST	<code>@PostMapping</code>	<code>/users</code>
PUT	<code>@PutMapping("/{id}")</code>	<code>/users/1</code>
PATCH	<code>@PatchMapping("/{id}")</code>	<code>/users/1?name=Updated Name</code>
DELETE	<code>@DeleteMapping("/{id}")</code>	<code>/users/1</code>



Handling request parameters, path variables, and request bodies.

- Handling Path Variables (@PathVariable)
 - Path variables are used to extract values from the URL path.
- Handling Request Parameters (@RequestParam)
 - Request parameters are used to pass optional or query parameters in the URL.
- Handling Request Body (@RequestBody)
 - Request bodies are used in POST, PUT, and PATCH requests to send JSON data.



Error Handling in REST APIs

- Custom exception handling in Spring Boot.
- Designing standard error responses.
- Implementing global exception handling using `@ControllerAdvice`.



Custom exception handling in Spring Boot.

- Spring Boot provides a structured way to handle exceptions in REST APIs using `@ExceptionHandler`, and `@RestControllerAdvice`
- Basic Exception Handling Using `@ExceptionHandler`
 - The `@ExceptionHandler` annotation is used inside a controller to handle specific exceptions.
- Global Exception Handling Using `@RestControllerAdvice`
 - Instead of handling exceptions per controller, you can use `@RestControllerAdvice` to centralize exception handling.



Designing standard error responses.

- A well-structured error response improves the API's usability and helps clients debug issues effectively.
- Structure of a Standard Error Response
- A standard error response should include the following fields:
 - timestamp → When the error occurred
 - status → HTTP status code (e.g., 400, 404, 500)
 - error → HTTP status message (e.g., "Bad Request", "Not Found")
 - message → A human-readable error message
 - path → The requested endpoint that caused the error

Implementing global exception handling using `@ControllerAdvice`



- Global exception handling allows you to manage errors centrally, improving maintainability and user experience.
- Why Use `@ControllerAdvice` or `@RestControllerAdvice`?
 - Centralized error handling for all controllers.
 - Ensures consistent error response format.
 - Separates business logic from error handling.



Advanced REST API Development



Securing REST APIs

- Authentication and Authorization: OAuth2 and JWT basics.
- Implementing security in Spring Boot using Spring Security.
- Protecting APIs from common vulnerabilities (e.g., CSRF, XSS).

Authentication and Authorization: OAuth2 and JWT basics.

- To secure a Spring Boot REST API use JWT (JSON Web Token) for authentication and authorization.
- Understanding Authentication vs. Authorization
 - **Authentication:** Verifies user identity (e.g., username & password).
 - **Authorization:** Grants or denies access to resources based on roles/permissions.
- What is JWT (JSON Web Token)?
 - JWT is a secure, compact token used for stateless authentication in REST APIs.
- Structure of a JWT Token: Header.Payload.Signature
 - Header: Contains token type (JWT) and signing algorithm (e.g., HS256).
 - Payload: Contains claims (user details, roles, expiration time, etc.).
 - Signature: Ensures the token is valid and untampered.



Implementing security in Spring Boot using Spring Security.

- Spring Security is a powerful framework that helps secure Spring Boot applications by handling authentication, authorization, and other security features.
- Enabling Spring Security
 - Edit pom.xml and add spring-boot-starter-security

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```
 - This will automatically secure all endpoints for application



Implementing security in Spring Boot using Spring Security.

- When you add spring-boot-starter-security, Spring Boot applies default security settings:
 - All endpoints require authentication.
 - The default user is "user", and the password is auto-generated at startup (found in the logs).
- To test this behavior: Start your application and check the logs for:
 - Using generated security password: abc123xyz
- You can override default user name and generated password

File: src/main/resources/application.properties

spring.security.user.name=admin

spring.security.user.password=test123



Protecting APIs from common vulnerabilities (e.g., CSRF, XSS).

- What is CSRF?
 - CSRF is an attack where a malicious website tricks users into making unwanted requests to your API while authenticated.
- Solution: Enable CSRF Protection
 - Spring Security enables CSRF protection by default.
 - However, for stateless APIs (JWT-based authentication), we usually disable CSRF since tokens protect against CSRF.
 - Default CSRF protection (for form-based authentication):
`http.csrf(csrf -> csrf.enable()) // Keep CSRF enabled for session-based authentication`
 - Disable CSRF for JWT-based authentication:
`http.csrf(csrf -> csrf.disable()) // Disable for JWT authentication`



Protecting APIs from common vulnerabilities (e.g., CSRF, XSS).

- Preventing XSS (Cross-Site Scripting)
- What is XSS?
 - XSS allows attackers to inject malicious JavaScript into web applications, which can steal sensitive information.
- Solution: Use Content Security Policy (CSP)
 - Spring Boot can set CSP headers to prevent XSS.
 - Modify your security configuration:

```
http.headers(headers -> headers
    .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'"))
)
```



Protecting APIs from common vulnerabilities (e.g., CSRF, XSS).

- Preventing SQL Injection
- What is SQL Injection?
 - SQL injection occurs when user inputs are directly concatenated into SQL queries, allowing attackers to execute harmful queries.
- Solution: Use Prepared Statements
 - Instead of:
`@Query("SELECT * FROM users WHERE username = '" + username + "'")`
 - Use:
`@Query("SELECT u FROM User u WHERE u.username = :username")`
`User findByUsername(@Param("username") String username);`
 - Spring Data JPA automatically prevents SQL injection when using `@Query` with parameters.



Protecting APIs from common vulnerabilities (e.g., CSRF, XSS).

- Secure API Endpoints with Proper CORS Configuration
- Why CORS?
 - CORS (Cross-Origin Resource Sharing) prevents unauthorized websites from making API calls.
- Solution: Configure Allowed Origins

```
public void addCorsMappings(CorsRegistry registry) {  
    registry.addMapping("/api/**")  
        .allowedOrigins("https://trusted.com")  
        .allowedMethods("GET", "POST", "PUT", "DELETE");  
}
```
- Only allow API calls from trusted domains.



Data Serialization and Validation

- Using Jackson for JSON serialization and deserialization.
- Validating API requests with `@Valid` and custom annotations.



Using Jackson for JSON serialization and deserialization.

- Jackson is the default JSON processing library used by Spring Boot for serializing Java objects to JSON and deserializing JSON to Java objects.
- Adding Jackson to a Spring Boot Project
 - Spring Boot automatically includes Jackson when using spring-boot-starter-web.
- JSON Serialization (Java Object → JSON)
 - When we return Java Object as response, Spring Boot automatically converts it to JSON.
- JSON Deserialization (JSON → Java Object)
 - When we send JSON data in a request, Spring Boot automatically converts it into a Java object.



Validating API requests with @Valid and custom annotations.

- Using @Valid for Request Validation
- Spring Boot integrates Jakarta Bean Validation (formerly Javax Validation) to validate API request payloads.
- Add Validation Dependency

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-validation</artifactId>  
</dependency>
```
- Validating Request Data with @Valid
 - Spring Boot uses @Valid to apply validation constraints defined in the model.



Validating API requests with @Valid and custom annotations.

- Creating Custom Validation Annotations
 - Spring Boot allows custom validation annotations when built-in ones are not enough.
- Example: Custom Annotation to Validate Product Name
 - Create the Annotation (@ValidProductName)
@Documented
@Constraint(validatedBy = ProductNameValidator.class)
@Target({ElementType.FIELD})
@Retention(RetentionPolicy.RUNTIME)
public @interface ValidProductName {
 String message() default "Product name must start with an uppercase letter";
 Class<?>[] groups() default {};
 Class<? extends Payload>[] payload() default {};
}



Validating API requests with @Valid and custom annotations.

- Create the Validator Class

```
public class ProductNameValidator implements ConstraintValidator<ValidProductName, String>
{
    @Override
    public boolean isValid(String name, ConstraintValidatorContext context) {
        return name != null && Character.isUpperCase(name.charAt(0));
    }
}
```

- Global Exception Handling for Validation Errors

- Spring Boot automatically throws MethodArgumentNotValidException when @Valid fails.
- To return a custom error response, use @RestControllerAdvice.



Practice Exercise

- Create REST API for Customers (id, name, email, address) using MS SQL Server or MySQL.
- Apply Custom Exception for Customer Not Found.
- Apply validation for name, and email field.



Optimizing REST APIs

- Pagination and filtering for large datasets.
- Caching responses to improve performance.
- Using asynchronous processing for long-running requests.



Pagination and filtering for large datasets.

- When handling large datasets in REST APIs, fetching all records at once can lead to performance issues and high memory usage.
- Pagination and filtering help optimize API responses, improving efficiency and user experience.
- Implementing Pagination & filtering in Spring Boot
 - Spring Boot supports pagination using Spring Data JPA's Pageable interface.
 - Add Spring Data JPA Dependency (If Missing)

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>
```



Caching responses to improve performance.

- Caching helps reduce database queries and response times by storing frequently accessed data in memory.
- Spring Boot provides caching support via Spring Cache Abstraction, with implementations like EhCache, Redis, Caffeine, and more.
- Enabling Caching in Spring Boot
 - Spring Boot requires enabling caching at the application level.
 - Add `@EnableCaching` in the Main Class
 - Caching API Responses using `@Cacheable`
 - `@Cacheable` stores method responses in cache so repeated calls return cached data instead of querying the database.
 - Apply Caching to the Service Layer



Caching responses to improve performance.

- Clearing Cache with `@CacheEvict`
 - When a product is added, updated, or deleted, we need to invalidate cache so new data is retrieved.
 - Cache is invalidated when a new record is added. The next request fetches fresh data from the database.
- Summary of Caching Annotations

Annotation	Description
<code>@EnableCaching</code>	Enables caching in the Spring Boot app
<code>@Cacheable(value = "cacheName")</code>	Caches method results
<code>@CacheEvict(value = "cacheName", allEntries = true)</code>	Clears cache on data update



Using asynchronous processing for long-running requests.

- Long-running requests can slow down API response times, leading to poor user experience and server overload.
- Asynchronous processing allows Spring Boot APIs to handle requests in the background while freeing up resources for other tasks.
- The `@Async` annotation allows methods to execute asynchronously in a separate thread.
- The method executes asynchronously without blocking the main thread.
- `CompletableFuture<String>` returns a future result when processing completes.



Testing and Documentation

- Writing unit tests for REST APIs using JUnit and Mockito.
- Automating API testing with Postman and REST Assured.
- Generating API documentation with Swagger/OpenAPI.

Writing unit tests for REST APIs using JUnit and Mockito.



- Unit testing ensures that individual components work as expected.
- For testing Spring Boot REST APIs, we use JUnit and Mockito to mock dependencies.
- Dependencies for Testing

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-test</artifactId>  
  <scope>test</scope>  
</dependency>
```
- spring-boot-starter-test includes JUnit and Mockito.

Automating API testing with Postman and REST Assured

- Automating API testing ensures the stability of your REST endpoints.
- Automating API Testing with Postman
 - Step 1: Install Postman
 - Download and install Postman.(<https://www.postman.com/downloads/>)
 - Step 2: Create a Collection
 - Open Postman → Click "New Collection" → Name it "Product API Tests".
 - Add requests like GET /products, POST /products, etc.
 - Step 3: Add a Test Script
 - Open the request (GET /v1/products).
 - Click "Tests" and add this script
 - Step 4: Run Automated Tests
 - Click Runner → Select "Product API Tests" → Run.
 - Postman Collection Runner executes multiple API tests at once.

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});
```

Automating API testing with Postman and REST Assured

- Automating API Testing with REST Assured (Java)
 - REST Assured is a Java library for testing REST APIs.
 - Add REST Assured Dependency

```
<dependency>  
  <groupId>io.rest-assured</groupId>  
  <artifactId>rest-assured</artifactId>  
  <scope>test</scope>  
</dependency>
```




Generating API documentation with Swagger/OpenAPI.

- Swagger (OpenAPI) is a powerful tool for documenting REST APIs, making them easy to understand, test, and consume.

- Add Swagger Dependencies

```
<dependency>  
  <groupId>org.springdoc</groupId>  
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>  
  <version>2.8.5</version>  
</dependency>
```

- Enable OpenAPI in Spring Boot

- Spring Boot automatically configures Swagger when the dependency is added.
- You can access the documentation at: <http://localhost:8080/swagger-ui/index.html>



Databases (Oracle / PostgreSQL / MySQL)



Introduction and Installation



Introduction and Installation

- Introduction and architecture concepts
- Installing PostgreSQL and using pgAdmin / psql
- Installing MySQL Server, Workbench, and command-line shell
- Setting up SQL Developer and connecting to Oracle Database



Introduction and architecture concepts

- A **database** is an organized collection of data that can be easily **accessed, managed, and updated**.
- A **Database Management System (DBMS)** is software that interacts with **users, applications, and the database itself**.
- Examples:
 - **PostgreSQL** – Open-source, advanced RDBMS
 - **MySQL** – Popular open-source relational DB
 - **Oracle Database** – Enterprise-grade commercial DB



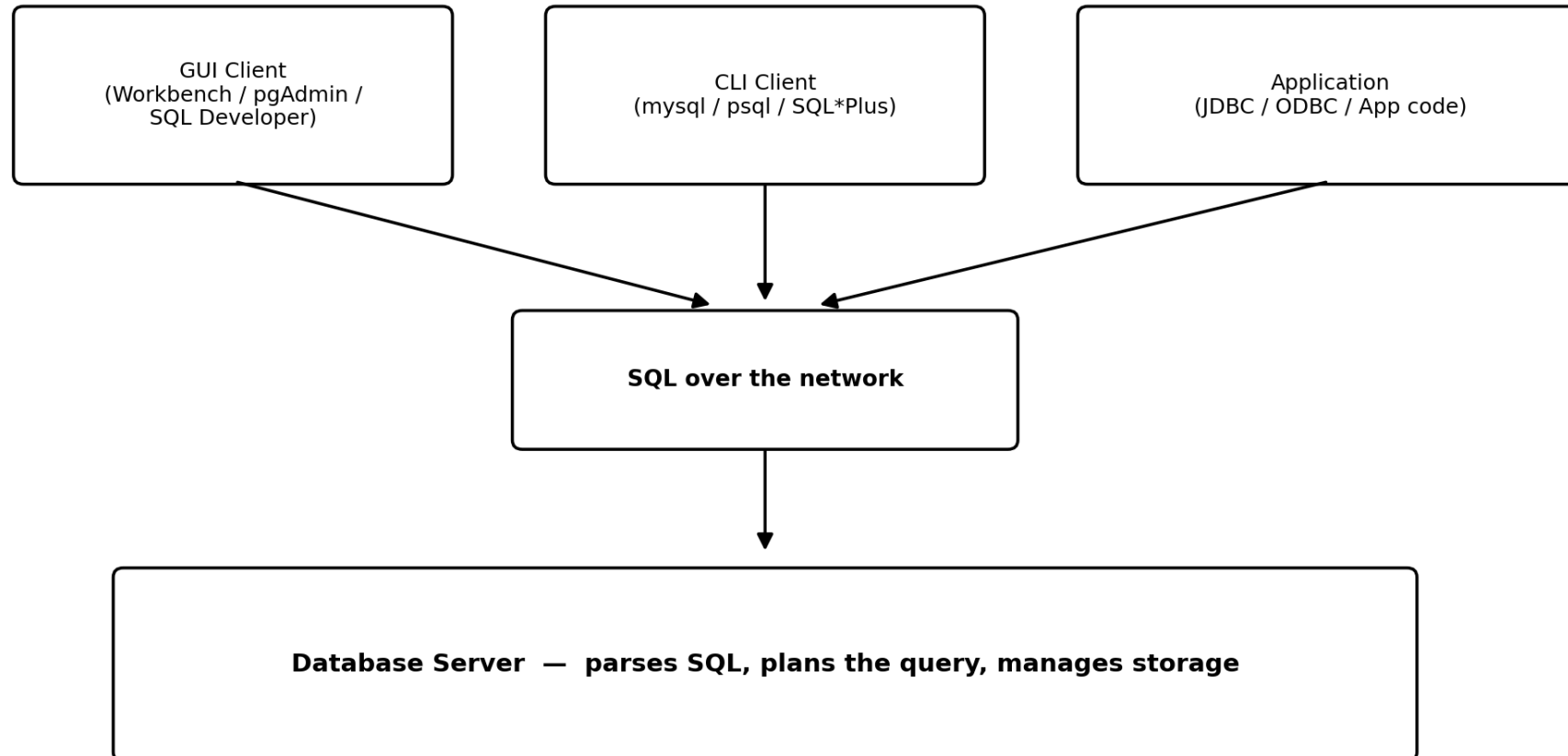
Introduction and architecture concepts

- Key Features of RDBMS
 - **Data Stored in Tables:** Data is structured in rows (records) and columns (fields).
 - **Relationships:** Establishes connections between tables using primary and foreign keys.
 - **SQL for Data Operations:** Standard language for querying and managing data.
 - **ACID Compliance:** Ensures **Atomicity, Consistency, Isolation, and Durability** in transactions.
 - **Data Integrity & Constraints:** Enforces rules like **Primary Key, Foreign Key, NOT NULL, UNIQUE, and DEFAULT** constraints.
 - **Scalability & Performance:** Supports indexing, partitioning, and replication for performance optimization.



Introduction and architecture concepts

Every Relational Database Follows the Same Client / Server Pattern



Installing PostgreSQL and using pgAdmin / psql

- Download from:
 - <https://www.postgresql.org/download/windows/>
- Run Installer:
 - Set password for postgres user
 - Default port: 5432
- Install Components:
 - PostgreSQL Server
 - pgAdmin
 - Command Line Tools (psql)



Installing MySQL Server, Workbench, and command-line shell

- Download My SQL Server and Workbench:
 - <https://dev.mysql.com/downloads/installer/>
- Choose:
 - Developer Default (recommended)
- Configure:
 - Root password
 - Port: 3306



Installing MySQL Server, Workbench, and command-line shell

- MySQL Workbench
 - GUI tool for:
 - Query execution
 - ER diagrams
 - DB design
- MySQL Command Line

```
mysql -u root -p
CREATE DATABASE training_db;
USE training_db;
```



Setting up SQL Developer and connecting to Oracle Database

- Oracle Database 23ai Free runs as a docker container

```
docker pull gvenzl/oracle-free:23-slim
docker run -d -p 1521:1521 -e ORACLE_PASSWORD=root --name demodb
gvenzl/oracle-free:23-slim
docker logs -f demodb
```
- SQL Developer from official website:
<https://www.oracle.com/database/sqldeveloper/>
Default Service Name: FREEPDB1
Username: SYSTEM
Password: root



Fundamental SQL Syntax



Fundamental SQL Syntax

- Working with numeric, string, and date data types
- Executing CRUD operations (INSERT, SELECT, UPDATE, DELETE)



Working with numeric, string, and date data types

- Numeric Data Types

Type	Description	Example
INT / INTEGER	Whole numbers	100
DECIMAL(p,s)	Fixed precision	1234.56
FLOAT / DOUBLE	Approximate values	123.4567

- String Data Types

Type	Description
VARCHAR(n)	Variable-length text
CHAR(n)	Fixed-length text
TEXT	Large text



Working with numeric, string, and date data types

- Date Data Types

Type	Description
DATE	Date only
TIME	Time only
TIMESTAMP	Date + Time



Executing CRUD operations (INSERT, SELECT, UPDATE, DELETE)

- CRUD = Create, Read, Update, Delete — the four basic things you do with data.
- INSERT adds a new row.
- SELECT reads rows back, optionally choosing only certain columns.
- UPDATE changes existing values; DELETE removes rows entirely.



Executing CRUD operations (INSERT, SELECT, UPDATE, DELETE)

- Example

```
INSERT INTO accounts (customer_id, account_type, balance)
VALUES (1, 'SAVINGS', 45000.00);
```

```
SELECT account_id, balance FROM accounts
WHERE account_type = 'SAVINGS';
```

```
UPDATE accounts SET balance = balance + 15000
WHERE account_id = 1;
```

```
DELETE FROM accounts WHERE account_id = 4;
```



Sorting & Filtering Data



Sorting & Filtering Data

- Filtering records using WHERE, operators, and pattern matching
- Ordering query results and using LIMIT for pagination

Filtering records using WHERE, operators, and pattern matching



- WHERE narrows down which rows a query returns.
- Combine conditions with AND, OR, and NOT.
- Comparison operators: =, <>, >, <, >=, <=.
- LIKE matches text patterns:
 - % means "any characters"
 - _ means "one character"

Filtering records using WHERE, operators, and pattern matching



- Example

```
SELECT * FROM transactions  
WHERE amount > 5000;
```

```
SELECT * FROM transactions  
WHERE txn_type = 'PAYMENT' AND status = 'FAILED';
```

```
SELECT * FROM transactions  
WHERE description LIKE '%bill%';
```



Ordering query results and using LIMIT for pagination

- The ORDER BY clause is used to sort query results based on one or more columns.
- ORDER BY sorts the result set — ASC (default) or DESC.
- Pagination shows results a page at a time instead of all at once.
- MySQL & PostgreSQL: LIMIT n OFFSET m.
- Oracle: OFFSET m ROWS FETCH FIRST n ROWS ONLY.



Ordering query results and using LIMIT for pagination

- Example:

```
SELECT transaction_id, amount, txn_date  
FROM transactions  
ORDER BY amount DESC;
```

-- Page 1 of 3 rows per page

```
SELECT * FROM transactions ORDER BY txn_date DESC LIMIT 3 OFFSET 0;
```

Oracle equivalent:

```
SELECT * FROM transactions ORDER BY txn_date DESC  
ORDER BY txn_date DESC OFFSET 0 ROWS FETCH FIRST 3 ROWS ONLY;
```



Database Architecture & Joins



Schema Design & Constraints



Schema Design & Constraints

- Defining Primary Keys, Foreign Keys, Unique, and Not Null constraints
- Establishing relationships (One-to-Many, Many-to-Many)
- Combining tables via INNER, LEFT, RIGHT, and CROSS JOINS



Defining Primary Keys, Foreign Keys, Unique, and Not Null constraints

- PRIMARY KEY — uniquely identifies every row in a table.
- FOREIGN KEY — links a row to a row in another table.
- UNIQUE — no two rows may share this value (e.g. email).
- NOT NULL — this column always requires a value.



Defining Primary Keys, Foreign Keys, Unique, and Not Null constraints

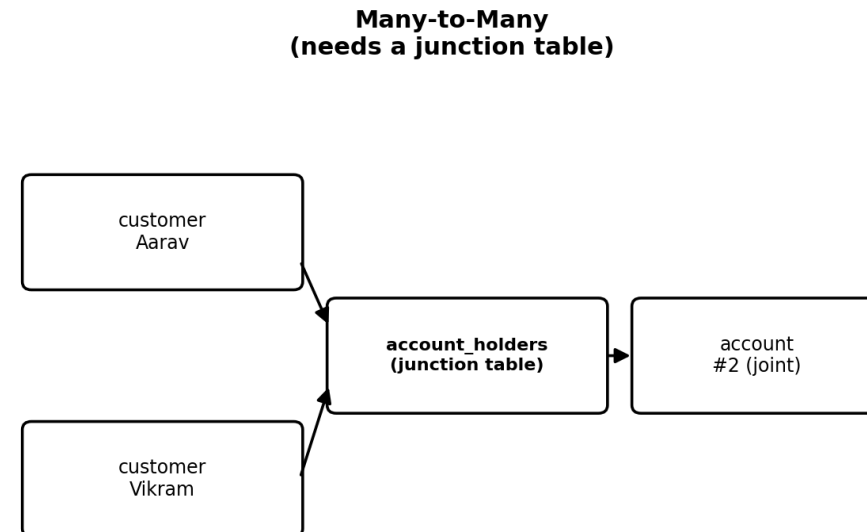
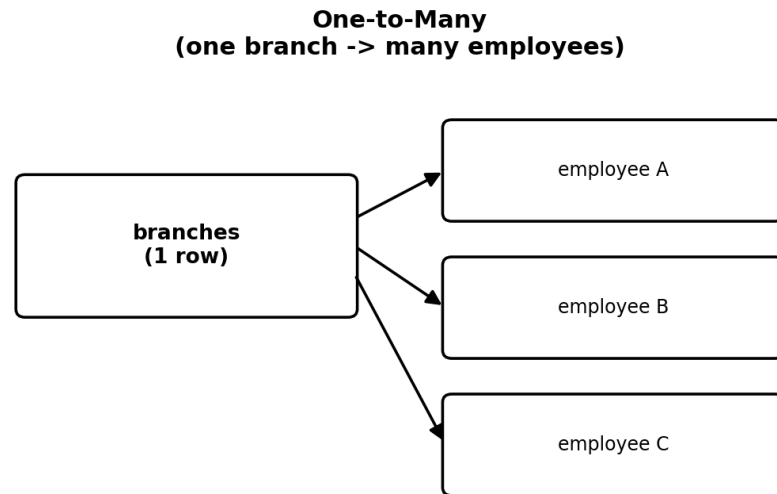
- Example:

```
CREATE TABLE accounts (  
    account_id INT PRIMARY KEY,  
    customer_id INT NOT NULL  
        REFERENCES customers(customer_id),  
    balance    DECIMAL(12,2) DEFAULT 0.00  
);
```



Establishing relationships (One-to-Many, Many-to-Many)

- One-to-many: put the foreign key on the "many" side — one branch has many employees.
- Many-to-many: needs its own junction table — a joint bank account links two customers to one account through `account_holders`.





Hands-on Activities

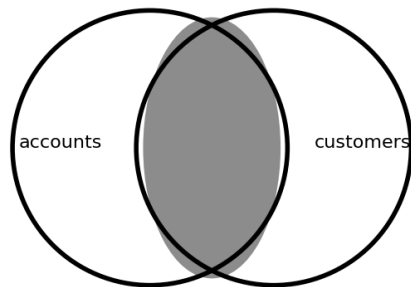
- Scenario: **E-Commerce System**
- You need to design a relational database schema for an e-commerce platform that manages:
 - Customers
 - Products
 - Orders



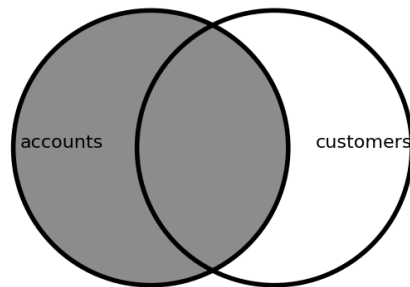
Combining tables via INNER, LEFT, RIGHT, and CROSS JOINS

- INNER JOIN — only rows that match in both tables.
- LEFT / RIGHT JOIN — all rows from one side, plus matches from the other (NULLs where there's no match).
- CROSS JOIN — every possible combination of rows from both tables.

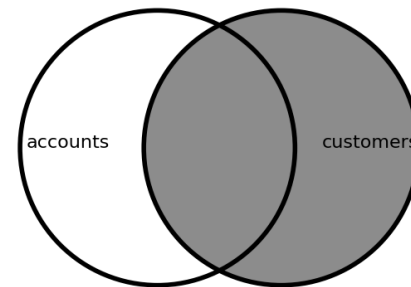
INNER JOIN



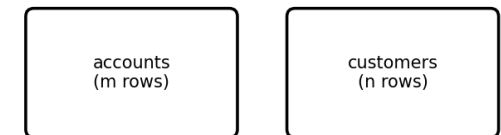
LEFT JOIN



RIGHT JOIN



CROSS JOIN



every combination
= $m \times n$ rows



Aggregation & Grouping



Aggregation & Grouping

- Summarizing data with COUNT, SUM, AVG, MIN, and MAX
- Restricting aggregated results using GROUP BY and HAVING
- Writing nested subqueries and derived tables



Summarizing data with COUNT, SUM, AVG, MIN, and MAX

- Aggregate functions summarize many rows into a single value.
 - COUNT(*) — how many rows.
 - SUM(), AVG() — total and average of a numeric column.
 - MIN(), MAX() — smallest and largest value.
- Example:

```
SELECT
    COUNT(*) AS total_txns,
    SUM(amount) AS total_amount,
    AVG(amount) AS avg_amount,
    MIN(amount) AS min_amount,
    MAX(amount) AS max_amount
FROM transactions
WHERE status = 'SUCCESS';
```

Restricting aggregated results using GROUP BY and HAVING

- GROUP BY groups rows that have the same values into summary rows.

- Syntax

```
SELECT column, AGG_FUNCTION(column)
FROM table_name
GROUP BY column;
```

- Example

```
SELECT dept_id, COUNT(*) AS total_employees
FROM employees
GROUP BY dept_id;
```

Restricting aggregated results using GROUP BY and HAVING

- HAVING filters grouped results (after aggregation)
- Difference:
 - WHERE → filters rows before grouping
 - HAVING → filters groups after grouping
- Example with HAVING

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 50000;
```
- Returns only departments with average salary > 50,000

Writing nested subqueries and derived tables



- A subquery is a query inside another SQL statement
- Example: Subquery in WHERE

```
SELECT name, salary  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
);
```

Writing nested subqueries and derived tables



- A derived table is a subquery used as a temporary table

- Example:

```
SELECT dept_id, avg_salary
FROM (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
) AS dept_avg
WHERE avg_salary > 50000;
```

- First calculates averages, then filters results



Performance & Optimization



Indexing & Optimization



Indexing & Optimization

- Creating B-Tree indexes to accelerate search performance
- Profiling slow queries with EXPLAIN statements



Creating B-Tree indexes to accelerate search performance

- Indexes help databases find data much faster—especially for large tables.
- The most common and default type is the B-Tree index.
- A **B-Tree (Balanced Tree)** index organizes data in a **sorted, hierarchical structure** so the database can quickly locate rows without scanning the entire table.
- Key Benefits
 - Fast lookups (WHERE column = value)
 - Efficient range queries (BETWEEN, <, >)
 - Supports sorting (ORDER BY)



Creating B-Tree indexes to accelerate search performance

- Creating a B-Tree Index

- Basic Syntax

```
CREATE INDEX index_name ON table_name (column_name);
```

- Example

```
CREATE INDEX idx_employee_salary ON employees (salary);
```

- Speeds up queries filtering or sorting by salary

Profiling slow queries with EXPLAIN statements

- When a query is slow, you need to understand how the database is executing it.
- That's exactly what EXPLAIN does—it shows the **execution plan** so you can diagnose performance issues.
- An execution plan describes:
 - How tables are accessed (index scan vs full scan)
 - Join methods used
 - Estimated cost and rows processed

Profiling slow queries with EXPLAIN statements

- Basic Usage of EXPLAIN

- PostgreSQL / MySQL

```
EXPLAIN SELECT * FROM employees WHERE salary > 50000;
```

- PostgreSQL (Detailed Execution)

```
EXPLAIN ANALYZE
```

```
SELECT * FROM employees WHERE salary > 50000;
```

- Oracle

```
EXPLAIN PLAN FOR
```

```
SELECT * FROM employees WHERE salary > 50000;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

- Actually runs the query and shows real execution time



Transactions & Storage Engines



Transactions & Storage Engines

- Comparing InnoDB vs. MyISAM storage engines
- Managing transactions (START TRANSACTION, COMMIT, ROLLBACK)



Comparing InnoDB vs. MyISAM storage engines

- In MySQL, a storage engine determines how data is stored, indexed, and managed.
- The two classic engines are InnoDB (default) and MyISAM (legacy).
- Key Features of InnoDB (Modern, Default Engine)
 - ACID transactions (safe commits/rollbacks)
 - Row-level locking → better concurrency
 - Foreign key support (enforces relationships)
 - Crash recovery (redo logs)
 - Clustered index (data stored with primary key)

Example

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(100)  
) ENGINE=InnoDB;
```




Comparing InnoDB vs. MyISAM storage engines

- Key Features of MyISAM (Older, Simpler Engine)

- Table-level locking (locks entire table)
- Faster reads in simple scenarios
- No transactions
- No foreign keys
- Stores data and indexes separately

Example

```
CREATE TABLE logs (  
    log_id INT PRIMARY KEY,  
    message TEXT  
) ENGINE=MyISAM;
```



Managing transactions (START TRANSACTION, COMMIT, ROLLBACK)

- Transactions ensure that a group of database operations either all succeed or all fail together, maintaining data integrity.
- This follows the ACID principles (Atomicity, Consistency, Isolation, Durability).
- Begins a new transaction.
`START TRANSACTION;`
- Saves all changes made during the transaction permanently.
`COMMIT;`
- Undoes all changes made during the transaction.
`ROLLBACK;`



Views & Stored Procedures



Views & Stored Procedures

- Creating Views for data security and simplification
- Writing Stored Procedures and basic triggers



Creating Views for data security and simplification

- A **view** is a virtual table based on a SQL query.
- It doesn't store data itself (in most cases) but presents data from one or more tables in a simplified and controlled way.
- Think of a view as:
 - A saved query
 - A layer of abstraction
 - A way to hide complexity or sensitive data

```
CREATE VIEW view_name AS  
SELECT column1, column2  
FROM table_name  
WHERE condition;
```



Writing Stored Procedures and basic triggers

- A stored procedure is a saved block of SQL code that you can execute whenever needed.
- Key Benefits
 - Reusable logic
 - Better performance (precompiled)
 - Improved security (restrict direct table access)



Writing Stored Procedures and basic triggers

- A trigger is a piece of code that runs automatically when a specific event happens on a table.
- Types of Triggers
 - BEFORE INSERT
 - AFTER INSERT
 - BEFORE UPDATE
 - AFTER UPDATE
 - BEFORE DELETE
 - AFTER DELETE



Apache Kafka



Kafka Core Concepts and Architecture



Introduction to Event Streaming



Introduction to Event Streaming

- Traditional messaging vs. Event Streaming Paradigms
- Core use cases(Log aggregation, stream processing, metrics)



Introduction to Kafka

- **Apache Kafka** is an open-source **distributed event streaming** platform used to build real-time data pipelines and event-driven applications.
- It is designed to handle **high volumes of data** with **high throughput, low latency, scalability, and fault tolerance**.



Introduction to Kafka

- **Core Concepts**

- **Producer** – Sends messages (events) to Kafka.
- **Consumer** – Reads messages from Kafka.
- **Topic** – A category or stream where messages are stored.
- **Partition** – A topic is split into partitions for parallel processing and scalability.
- **Broker** – A Kafka server that stores and serves data.
- **Consumer Group** – Multiple consumers that share the work of reading messages from a topic.
- **Offset** – The position of a message within a partition.



What is an Event?

- An **event** is a record of something that happened in a system.
- Examples
 - Payment processed
 - Transaction failed
- Each event typically contains:
 - Event name (e.g., PaymentCompleted)
 - Timestamp
 - Data (payload) (amount, userId, etc.)



What is Event Streaming?

- **Event streaming** is a modern data architecture pattern where events (changes in state) are continuously produced, captured, stored, and processed in real time.
- Event streaming is the process of:
 1. Capturing events as they occur
 2. Storing them in a durable system
 3. Processing them in real-time or near real-time
 4. Delivering them to multiple consumers
- It enables systems to react instantly to data changes.



Traditional messaging vs. Event Streaming Paradigms

- **Traditional Messaging (Message Queue)**
 - Traditional messaging systems (like queues) are designed to distribute tasks between producers and consumers.
- Flow
 - Producer → Queue → Consumer
- Key Characteristics
 - Point-to-point communication
 - Message is consumed once
 - Message is **removed** after processing
 - Focus on task execution (work distribution)
 - Typically pull-based (consumer polls the queue)



Traditional messaging vs. Event Streaming Paradigms

- **Traditional Messaging (Message Queue)**
- Example
 - A payment service sends a message to a queue: `ProcessPayment`
 - One worker service consumes it and processes the payment
 - Message is removed after processing



Traditional messaging vs. Event Streaming Paradigms

- **Event Streaming Paradigm**

- Event streaming systems are designed to capture and distribute events (facts) in real time to multiple consumers.

- Flow

- Producer → Event Broker (Kafka) → Multiple Consumers

- Key Characteristics

- Publish-subscribe model
- Events are immutable (append-only)
- Events are retained for a configurable time
- Multiple consumers can read the same event independently
- Supports real-time streaming and replay
- Loosely coupled systems



Traditional messaging vs. Event Streaming Paradigms

- **Event Streaming Paradigm**
- Example
 - A payment event is published: `PaymentCompleted`
 - Multiple services consume it:
 - Fraud detection
 - Notification
 - Analytics
 - Ledger update
 - All services process the same event independently.



Core use cases (Log aggregation, stream processing, metrics)

- **Log Aggregation**

- Log aggregation is the process of collecting logs from multiple systems/services into a central platform for monitoring and analysis.

- **Traditional Problem**

- Logs scattered across multiple servers
- Difficult to debug issues
- No centralized visibility

- **Event Streaming Solution**

- All applications publish logs as events to a central system (Kafka topic).

App1 →

App2 → Kafka (logs topic) → ELK / Splunk / Monitoring Tools

App3 →



Core use cases (Log aggregation, stream processing, metrics)

- **Stream Processing**

- Stream processing is the ability to process data continuously as it flows in real time, instead of storing and processing later.

- **Key Idea**

- Process events as they arrive, not in batches.

Producer → Kafka → Stream Processor → Output

- **Example : Fraud Detection System**

1. User makes a transaction
2. Event sent: TransactionInitiated
3. Stream processor checks:
 - Transaction amount
 - Location mismatch
 - Frequency
4. If suspicious → trigger alert instantly



Core use cases (Log aggregation, stream processing, metrics)

- **Metrics & Monitoring**

- Metrics are numerical measurements used to monitor system performance and business KPIs.

- **Examples of Metrics**

- Transactions per second
- API response time
- Error rate
- Active users

- **Event Streaming Approach**

- Applications continuously emit metrics as events:
Services → Kafka (metrics topic) → Prometheus / Grafana



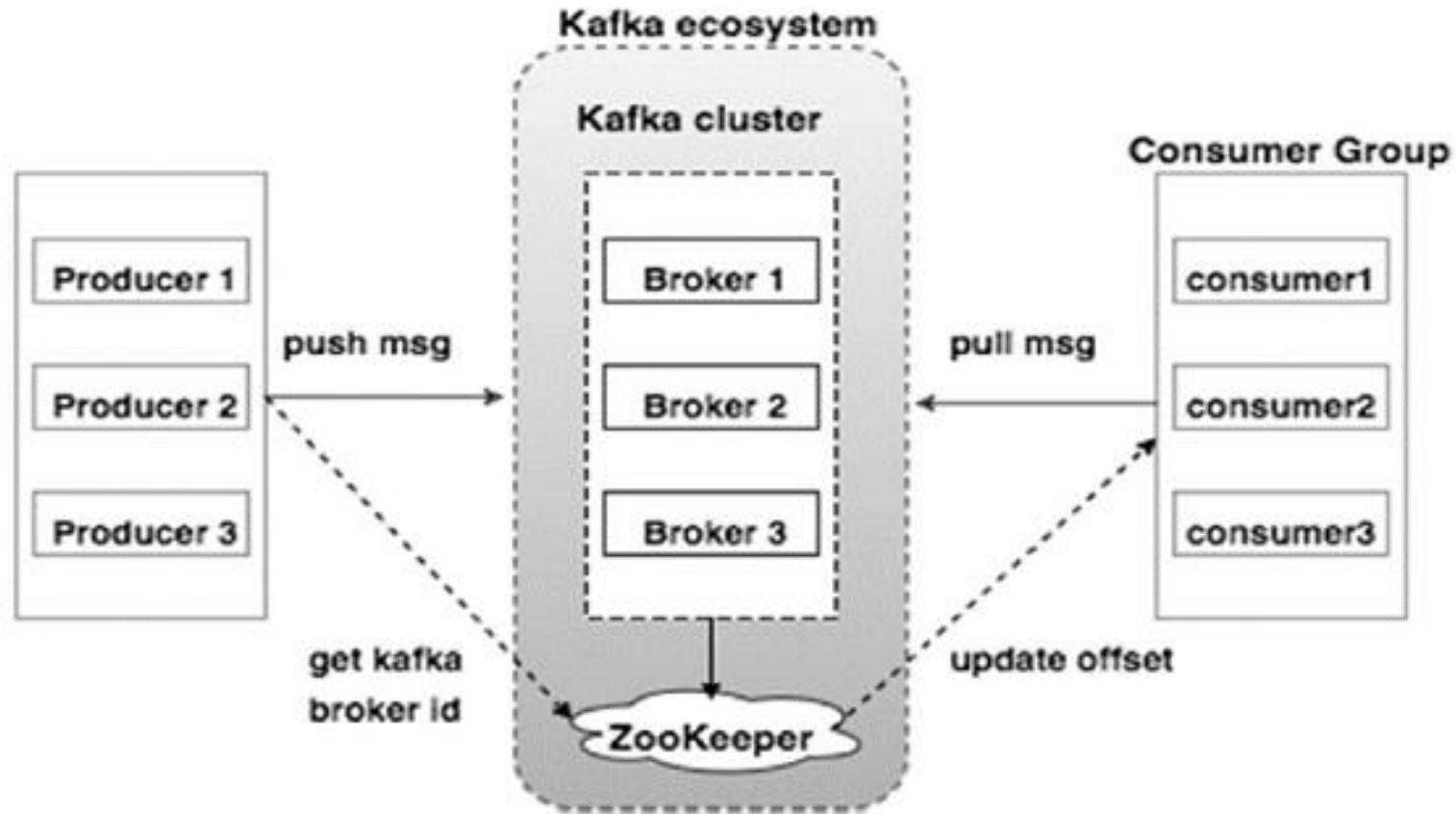
Core Architectural Components



Core Architectural Components

- Understanding Brokers, Clusters and ZooKeeper / KRaft Mode
- Topics, Partitions and Segment files

Understanding Brokers, Clusters and ZooKeeper / KRaft Mode



Understating Brokers, Clusters and ZooKeeper / KRaft Mode



- A **Kafka Broker** is a single Kafka server instance responsible for:
 - Receiving messages from producers
 - Storing data (topics & partitions)
 - Serving data to consumers
- Key Responsibilities
 - Handles read/write requests
 - Stores topic partitions
 - Manages replication
 - Maintains offsets

Understating Brokers, Clusters and ZooKeeper / KRaft Mode



- **Kafka Cluster**

- A Kafka Cluster is a group of brokers working together.

- **Why Cluster?**

- Scalability
- Fault tolerance
- High availability

- **Concepts Inside Cluster**

1. **Partitioning**

- Topics are split into partitions
- Enables parallel processing

2. **Replication**

- Each partition has:
- Leader → handles read/write
- Followers → replicate data

3. **Leader Election**

- If leader fails → another broker becomes leader

Understating Brokers, Clusters and ZooKeeper / KRaft Mode



- ZooKeeper (Legacy Coordination Layer)
 - ZooKeeper is a centralized service used (in older Kafka versions) for cluster coordination and metadata management.
 - ZooKeeper is being removed (Kafka 2.8+ onwards)
 - Replaced by **KRaft (Kafka Raft Metadata mode)**
- KRaft Benefits
 - No external dependency
 - Simpler architecture
 - Better scalability
 - Faster leader election



Topics, Partitions and Segment files

- **Topics**

- A topic is a logical category or stream where messages (events) are published.
- Similar to a table in a database or a channel
- Producers write to topics
- Consumers read from topics

- **Example**

- transactions
- payments
- user-activity
- Producer → Topic (transactions) → Consumer

- **Key Points**

- Topics are append-only logs
- Messages are immutable (cannot be changed)
- Each message has an offset



Topics, Partitions and Segment files

- **Partitions**

- A partition is a subset of a topic that allows Kafka to scale horizontally.
- Instead of storing all data in one place, Kafka splits a topic into multiple partitions.

- **Why Partitions?**

- Parallel processing
- High throughput
- Scalability

- **Structure**

Topic: transactions

Partition 0: [M1, M2, M3]

Partition 1: [M4, M5, M6]

Partition 2: [M7, M8, M9]



Topics, Partitions and Segment files

- **Partitions - Key Concepts**
- Ordering Guarantee
 - Order is guaranteed only within a partition
 - Not across partitions
- Offset
 - Each message has a unique offset within its partition
 - Partition 0:
 - Offset 0 → M1
 - Offset 1 → M2
 - Offset 2 → M3
- Key-based Partitioning
 - Kafka uses a key to decide which partition a message goes to
 - Same key → same partition → preserves order



Topics, Partitions and Segment files

- Segment Files (Kafka Storage Internals)
 - Kafka does not store partitions as a single large file.
 - Instead, each partition is split into smaller chunk files called segment files.
- Why Segment Files?
 - Efficient disk usage
 - Faster reads/writes
 - Easier data cleanup (retention policies)
- Components of Segment
 - .log file
 - Stores actual messages
 - .index file
 - Maps offsets → physical location in log file

Structure on Disk

Topic: transactions

Partition 0:

00000000000000.log

00000000000000.index

00000000000001.log

00000000000001.index



Producers and Consumers



Producers and Consumers

- How producers publish data and handle partitioning strategies
- Consumer groups, offsets and message delivery semantics



How producers publish data and handle partitioning strategies

- **Kafka Producer**

- A producer is an application that publishes (writes) events/messages to Kafka topics.

- **Basic Flow**

Producer → Topic → Partition → Kafka Broker

- **What a Producer Sends**

- **Each message typically contains:**

- Topic name
 - Key (optional)
 - Value (actual data)
 - Timestamp (auto or custom)



How producers publish data and handle partitioning strategies

- Step-by-Step
 1. Producer connects to Kafka cluster
 2. Sends message to a **topic**
 3. Kafka decides **which partition** to store the message in
 4. Message is appended to the partition log



Consumer groups, offsets and message delivery semantics

- **Kafka Consumer**

- A consumer reads messages from Kafka topics.

Kafka Topic → Consumer → Process Data

- **Consumer Groups**

- Multiple consumers form a consumer group
 - Each partition is consumed by only one consumer in the group

Partition 0 → Consumer 1

Partition 1 → Consumer 2

Partition 2 → Consumer 3



Consumer groups, offsets and message delivery semantics

- **Offsets**

- An offset is a unique identifier for each message within a partition.

Partition 0:

Offset 0 → Msg1

Offset 1 → Msg2

Offset 2 → Msg3

- **Why Offsets Matter**

- Tracks which messages are consumed
 - Enables:
 - Resume processing
 - Replay events
 - Fault tolerance



Consumer groups, offsets and message delivery semantics

- **Message Delivery Semantics**

- This defines how reliably messages are delivered and processed.

- 1. At Most Once**

- Message is processed zero or one time
 - Offset committed before processing
 - Commit → Process
 - Risk: Message loss
 - Use Case: Logging, non-critical metrics



Consumer groups, offsets and message delivery semantics

- **Message Delivery Semantics**

- This defines how reliably messages are delivered and processed.

2. **At Least Once**

- Message is processed one or more times
 - Offset committed after processing

Process → Commit

- Risk: Duplicate processing
 - Use Case: Financial transactions



Consumer groups, offsets and message delivery semantics

- **Message Delivery Semantics**

- This defines how reliably messages are delivered and processed.

3. **Exactly Once**

- Message is processed only once
 - Achieved using:
 - Transactions
 - Use Case: Critical systems (payments, banking)



Stream Processing and Ecosystems



Kafka Stream API



Kafka Stream API

- Building real-time stream processing applications
- Stateless vs. Stateful transformations (Joins, Aggregations, Windowing)



Building real-time stream processing applications

- Example: Fraud Detection System
- Step 1: Input
 - Topic: transactions
- Step 2: Processing Logic
 - Filter high-value transactions
 - Group by user
 - Count transactions in last 1 minute
 - If threshold exceeded → fraud alert
- Step 3: Output
 - Topic: fraud-alerts



Stateless vs. Stateful transformations (Joins, Aggregations, Windowing)

- Stateless: Each message is processed independently
- Stateful: Processing depends on past data (state/memory)
- Stateless Transformations
 - No memory of previous events
 - Each record is processed in isolation
 - Fast and simple
- Common Stateless Operations
 - filter()
 - map()
 - flatMap()
 - peek()



Stateless vs. Stateful transformations (Joins, Aggregations, Windowing)

- Stateful Transformations
 - Requires memory of past events
 - Uses state stores (RocksDB)
 - Enables complex analytics
- Core Stateful Operations
 - Aggregations (count, reduce, aggregate)
 - Joins
 - Windowing



Stateless vs. Stateful transformations (Joins, Aggregations, Windowing)

- Aggregations
 - Combine multiple events into a single result
- Example

```
stream
    .groupByKey()
    .count();
```

 - Counts number of events per key



Stateless vs. Stateful transformations (Joins, Aggregations, Windowing)

- Joins
 - Combines data from two streams or tables
- Types of Joins
 1. Stream-Stream Join
 - Join two event streams
 - Requires time window
 - transactions + user-clicks
 - Example: Match payment with login activity within 5 mins



Stateless vs. Stateful transformations (Joins, Aggregations, Windowing)

- Joins
 - Combines data from two streams or tables
 - Types of Joins
- ## 2. Stream-Table Join
- Stream joined with latest state (KTable)
transactions + user-profile
 - Example: Add user info to transaction



Stateless vs. Stateful transformations (Joins, Aggregations, Windowing)

- Joins
 - Combines data from two streams or tables
- Types of Joins
- 3. Table-Table Join
 - Join two KTables (state-to-state)
- Key Insight
 - Joins are stateful because Kafka must store data temporarily



Stateless vs. Stateful transformations (Joins, Aggregations, Windowing)

- Windowing (Critical Concept)
 - Streams are infinite → we need time boundaries

- Types of Windows

1. Tumbling Window

- Fixed, non-overlapping
[00:00–00:05], [00:05–00:10]

2. Hopping Window

- Overlapping windows
Every 5 min, window size 10 min

3. Sliding Window

- Based on event timing differences

Example

```
.groupByKey()  
.windowedBy(TimeWindows.of(Duration.ofMinutes(5)))  
.count();
```

- Count events every 5 minutes



KSQL/ksqlDB



KSQL/ksqlDB

- Querying real-time event streams using familiar SQL syntax
- Creating materialized views and streaming pipelines



Querying real-time event streams using familiar SQL syntax

- ksqlDB is a streaming SQL engine for Apache Kafka.
- It allows you to:
 - Query Kafka topics in real time
 - Transform streams using SQL syntax
 - Build streaming pipelines without Java code
- Think of it like:
 - Kafka Topics + SQL = ksqlDB



Querying real-time event streams using familiar SQL syntax

- Continuous Queries

- Unlike traditional SQL:
 - Queries never stop
 - Results update continuously

- Example

```
SELECT * FROM transactions EMIT CHANGES;
```

- Streams live data continuously



Querying real-time event streams using familiar SQL syntax

- Filtering Data

```
SELECT *  
FROM transactions  
WHERE amount > 10000  
EMIT CHANGES;
```

- Real-time high-value transaction detection

- Transformations

```
SELECT userId, amount * 0.1 AS tax  
FROM transactions  
EMIT CHANGES;
```



Querying real-time event streams using familiar SQL syntax

- Aggregations (Stateful Processing)
- Example: Count Transactions Per User

```
SELECT userId, COUNT(*)  
FROM transactions  
GROUP BY userId  
EMIT CHANGES;
```

- This is stateful



Creating materialized views and streaming pipelines

- A materialized view in ksqlDB is a continuously updated table that stores the result of a query.
- Unlike traditional SQL:
 - It updates in real time
 - It is backed by Kafka topics + state stores
- Think of it like:
Stream of Events → Query → Stored, Updated View (Table)
- Example Use Case
 - Total spending per user
 - Active users in last 5 minutes
 - Fraud detection counters



Creating materialized views and streaming pipelines

- Streaming Pipelines
 - A series of transformations where data flows through multiple steps in real time.
- Pipeline Flow
 - Input Topic → Stream → Transform → Output Topic → Another Stream → Final Output



Performance Tuning & Security



Performance Tuning & Security

- Optimizing throughput, latency, and compression (Snappy, lz4)
- Securing clusters with SSL/TLS encryption, SASL authentication, and ACLs



Optimizing throughput, latency, and compression (Snappy, lz4)

- Getting Kafka to perform well is all about balancing throughput (speed of data movement) and latency (delay), while ensuring secure, reliable delivery
- Core Performance Goals

Metric	Meaning
Throughput	Messages/sec Kafka can handle
Latency	Time from producer → consumer
Durability	Data safety guarantees

- Improving one often affects the others (trade-offs!)



Optimizing throughput, latency, and compression (Snappy, lz4)

- Compression reduces network + disk usage
- Supported Types

Compression	Speed	Compression Ratio	Use Case
none	Fastest	None	Low latency
snappy	Fast	Medium	Balanced (most common)
lz4	Very Fast	Medium	High throughput systems
gzip	Slow	High	Storage optimization



Securing clusters with SSL/TLS encryption, SASL authentication, and ACLs

- Production Kafka must be secured on three layers:
Encryption (SSL/TLS) → Authentication (SASL) → Authorization (ACLs)
- 1. SSL/TLS Encryption (Data Protection)
 - Encrypts data in transit
 - Prevents eavesdropping & MITM attacks
- How it Works
 - Producer ⇔ SSL ⇔ Broker ⇔ SSL ⇔ Consumer
- Key Components
 - Keystore → Your identity (certificate + private key)
 - Truststore → Trusted certificates



Securing clusters with SSL/TLS encryption, SASL authentication, and ACLs

2. SASL Authentication (Identity Verification)

- Verifies who is connecting
- Supported Mechanisms

Mechanism	Description
PLAIN	Username/password (simple, use with SSL)
SCRAM	Secure hashed credentials (recommended)
GSSAPI (Kerberos)	Enterprise-grade
OAUTHBEARER	Token-based (modern systems)

- Best Practice
 - Always combine: SASL + SSL (never SASL alone)



Securing clusters with SSL/TLS encryption, SASL authentication, and ACLs

3. Authorization with ACLs (Access Control)

- Controls who can do what
- Types of Permissions
 - READ
 - WRITE
 - CREATE
 - DELETE
 - ALTER



Redis Cache



Spring Boot Redis Integration



Spring Data Redis Basics



Spring Data Redis Basics

- Connecting Spring Boot to Redis using Lettuce or Jedis clients
- Using RedisTemplate to perform simple CRUD operations
- Configuring basic text and JSON serializers for Java objects



Spring Data Redis Basics

- Redis cache is an in-memory key-value data store commonly used to cache frequently accessed data, reducing database load and improving application performance.
- How Redis Cache Works
 1. A client requests data.
 2. The application checks Redis first.
 3. If the data exists (cache hit), Redis returns it immediately.
 4. If the data doesn't exist (cache miss), the application fetches it from the database, stores it in Redis, and returns it to the client.



Spring Data Redis Basics

- Why Use Redis for Caching?
- Very fast: Data is stored in RAM (microsecond latency).
- Reduces database load: Fewer database queries.
- Improves response time: Faster API and web application performance.
- Supports expiration (TTL): Automatically removes stale cache entries.
- Scalable: Can handle millions of requests per second.

Connecting Spring Boot to Redis using Lettuce or Jedis clients



- Spring Boot integrates Redis using Spring Data Redis, which provides:
 - High-level abstraction over Redis operations
 - Template-based API (RedisTemplate)
 - Repository support (similar to JPA)
 - Cache abstraction (@Cacheable)

- Dependency (Maven)

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-data-redis</artifactId>  
</dependency>
```

Connecting Spring Boot to Redis using Lettuce or Jedis clients



- Connecting Spring Boot to Redis
 - Spring Boot uses Lettuce by default.
 - No extra config needed. Spring Boot auto-configures.
- application.properties

```
spring.redis.host=localhost
spring.redis.port=6379
spring.redis.timeout=60000
```

Using RedisTemplate to perform simple CRUD operations



- Spring Data Redis provides two main approaches:
- RedisTemplate (Low-level, flexible)
- Repository abstraction (High-level, like JPA)



Configuring basic text and JSON serializers for Java objects

- Redis stores bytes, not Java objects. So Spring needs serializers to:
 - Convert Java → bytes (serialization)
 - Convert bytes → Java (deserialization)
- Types of Serializers

Serializer	Use Case
StringRedisSerializer	Keys / simple text values
GenericJackson2JsonRedisSerializer	JSON (recommended)
Jackson2JsonRedisSerializer	Typed JSON
JdkSerializationRedisSerializer	Default (not recommended)



Happy Learning :)